

Problem 1

The *score* of a sequence of integers $b_1, b_2, \dots, b_\ell$ is defined as

$$\sum_{1 \leq i < \ell} (b_i - b_{i+1})^2.$$

Describe and analyze an $O(n^2k)$ -time dynamic programming algorithm to compute the highest possible score over all subsequences of length at most k in a given array $A[1..n]$ of integers, where k is a given parameter. For example, $1, 9, 3$ is a subsequence of $[3, \underline{1}, 5, 6, \underline{9}, 7, 4, \underline{3}, 8]$ of length 3 with score $(1 - 9)^2 + (9 - 3)^2 = 64 + 36 = 100$, which is the highest-scoring 3-length subsequence in the array. Note that the items in a subsequence need not be contiguous in the original array.

Solution: Let $HCS(i, j, \ell)$ be the score of the highest-scoring subsequence of $A[i..n]$ of length at most $\ell \geq 0$ that begins with $A[j]$, where $i > j$. $\max_{1 \leq i \leq n} HCS(i + 1, i, k)$ solves the given problem.

$$HCS(i, j, \ell) = \begin{cases} 0 & \text{if } i > n \text{ or } \ell = 0 \\ \max \begin{cases} (A[i] - A[j])^2 + HCS(i + 1, i, \ell - 1), \\ HCS(i, j + 1, \ell) \end{cases} & \text{otherwise} \end{cases}$$

We next use dynamic programming to obtain an efficient algorithm. We memoize the function in a $n \times n \times (k + 1)$ array M , where $M[i][j][\ell]$ stores $HCS(i, j, \ell)$. The evaluation order is given by the following nested for-loops:

```
for j from n downto 1:
  for i from n downto j + 1:
    for ℓ from 0 to k:
```

There are $O(n^2k)$ subproblems and each is computed in $O(1)$ time with memoization, so the overall runtime is $O(n^2k)$.

Solution: (An alternative recurrence.) Let $HCS(i, \ell)$ be the score of the highest-scoring subsequence of $A[i..n]$ of length at most $\ell \geq 0$ that begins with $A[i]$. $\max_{1 \leq i \leq n} HCS(i, k)$ solves the given problem.

$$HCS(i, \ell) = \begin{cases} 0 & \text{if } i \geq n \text{ or } \ell = 0 \\ \max_{i < j \leq n} \{ (A[i] - A[j])^2 + HCS(j, \ell - 1) \} & \text{otherwise} \end{cases}$$

We next use dynamic programming to obtain an efficient algorithm. We memoize the function in a $n \times (k + 1)$ array M , where $M[i][\ell]$ stores $HCS(i, \ell)$. The evaluation order is given by the following nested for-loops:

```
for i from n downto 1:
  for ℓ from 0 to k:
```

There are $O(nk)$ subproblems and each is computed in $O(n)$ time with memoization, so the overall runtime is $O(n^2k)$.

Problem 2

The sequel to the worldwide classic board game *Snakes and Ladders* is here: *Just Ladders*! The game of *Just Ladders* is not a race to the finish nor does it involve any randomness by way of dice rolling or spinner spinning; instead, it is a game about collecting as many points as possible by the time your token reaches the end of the board.

A game of *Just Ladders* consists of a $\sqrt{k} \times \sqrt{k}$ board, where $k > 0$ is a perfect square, that has $n \geq 1$ “ladders.” The spaces are numbered so that there is a path along adjacent spaces starting from space 1 to space k , and each space i may have at most one ladder leading up from that space to a higher space j , $j > i$. A ladder leading up from a space i is associated with a number of points, $p_i > 0$. Note that a space may have many ladders leading into it but at most one leading up from it.

As a player of the game, you begin with your token on space 1. Then you repeat the following until your token reaches the last space on the board, space n :

1. You can always move your token from its current space i to a later space $j > i$, for no gain in points.
2. Additionally, if your current space i has a ladder *leading up* from that space to another space $j > i$, you can use the ladder to move your token to space j and gain p_i points for using it.

Formally, a board for *Just Ladders* consists of a perfect square integer k , denoting the size of the board, and three arrays $B[1..n]$, $T[1..n]$, and $P[1..n]$, that represent the $n \geq 1$ ladders: B is a sorted (potentially non-contiguous) subsequence of $[1, 2, \dots, n]$ such that there is a ladder leading up from space $B[i]$ to space $T[i]$, $B[i] < T[i]$, that awards $P[i]$ points for using the ladder, for each i from 1 to n .

Design and analyze an efficient dynamic programming to compute the maximum number of points that is possible to obtain given board size k and arrays B , T , and P . The target runtime is $O(n \log n)$. [Hint: Don't aim for the target runtime from the start. First obtain a **correct** solution, and **then** consider how to improve the runtime.]

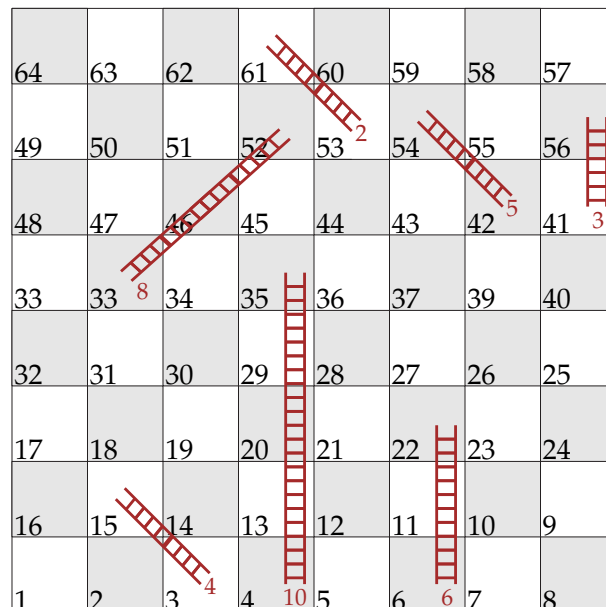


Figure 1. Example of a $n = 64$ space *Just Ladders* board with $k = 7$ ladders. The maximum possible points is 16, obtained by first moving to space 6 and taking the ladder to space 22 gaining 6 points, then moving to space 33 and taking the ladder to space 52 gaining 8 points, then moving to space 53 and taking the ladder to space 61 gaining 2 points, and finally moving to space 64 gaining 0 points.

Solution: Note that there is no opportunity to gain any points before the space $B[1]$, starting from the first space (which may be $B[1]$). Let $MS(i)$ denote the maximum score obtainable in *Just Ladders* starting at board space $B[i]$. We want to find $MS(1)$.

We can either take the i -th ladder or not. If we choose not to take it, we want the best score possible using the remaining ladders $n - i$ ladders, and the next opportunity to possibly gain points is when we reach space $B[i + 1]$, the bottom of the next ladder. If we take the i -th ladder, we gain $P[i]$ points then want the best score possible using the ladders whose bottom space is at or after the top space of the i -th ladder, $T[i]$. That is, if $X(i)$ is the minimum index such that $B[X(i)] \geq T[i]$ (if there is no such index, set $X(i) = n + 1$), we want the maximum score starting from space $B[X(i)]$, the next space at which there is an opportunity to gain points (by taking ladder $B[X(i)]$ or moving past it). Thus, MS is described by the following recurrence:

$$MS(i) = \begin{cases} 0 & \text{if } i > n \\ \max\{MS(i + 1), P[i] + MS(B[X(i)])\} & \text{otherwise} \end{cases}$$

We compute $X(i)$ for each $i \in \{1, \dots, n\}$ using binary search on the array B , as its entries are sorted. This takes $O(n \log n)$ time in total.

We now use dynamic programming. We use a 1D array $M[0..n]$ as our memoization data structure, where we store $MS(i)$ in $M[i]$ for each i . We fill M in decreasing order, from $i = n$ to 1. There are $O(k)$ subproblems in total, and evaluating each one (after preprocessing the $X(i)$'s) takes $O(1)$ time with memoization. Therefore the overall runtime is $O(n \log n)$, dominated by the time to compute the $X(i)$'s. As mentioned above, to solve the original problem, we compute $MS(1)$.

Problem 3

The corporate structure at $\mathbb{Y}$, a recently-renamed social media company, can be represented by a rooted tree, where the employees are nodes and the child nodes of an employee are other employees that directly report to them. The CEO, at the root of the tree, does not report to anyone. The CEO wishes to pick a subset of employees to attend a publicity event so that, for each employee, either that employee or to whom that employee reports must attend the event. However, no employee wants to attend the event, and they are only willing to go if they are paid extra. Each employee has a price (\$) they are willing to be paid in order to attend the event (including the CEO). Your task is to select a subset of employees to attend the event so that, for any employee, either that employee attends or the employee to whom they report attends (possibly both), while paying the minimum total dollars.

Formally, let T be a rooted binary tree with n nodes that represents the corporate structure, where the CEO correspond to the root node, $c \in T$. Each employee (node) $v \in T$ stores the price $p(v) \geq 0$ that the employee is willing to accept to attend the event, and the list $\ell(v)$ of $r(v)$ employees (nodes) that report directly to employee v . Any employee v without other employees that report to them has $r(v) = 0$ and empty list $\ell(v)$. Describe an efficient dynamic programming algorithm that computes a suitable subset of employees to attend the event that has minimum total price. The target runtime is $O(n)$. [Hint: Compare and contrast this problem with the maximum independent set (MIS) problem from lecture and our textbook. Don't aim for the target runtime from the start. First obtain a **correct** solution, and **then** consider how to improve the runtime.]

Solution: For a vertex $v \in T$, let $g(u)$ be the set of grandchildren of u in T ; that is,

$$g(u) = \bigcup_{e \in r(u)} r(e).$$

Let $MPE(u)$ be the minimum total price of a suitable guest list among the employees in the subtree rooted at employee u , where, if u is not the CEO, then the parent of u is already included in the guest list. The function is described by the following recurrence:

$$MPE(u) = \begin{cases} 0 & u \text{ is leaf} \\ \min \begin{cases} p(u) + \sum_{v \in r(u)} MPE(v) \\ \sum_{v \in r(u)} p(u) + \sum_{w \in g(u)} MPE(w) \end{cases} & \text{otherwise} \end{cases}$$

The top case in the min function corresponds to the decision to include u in the event list, so their children may or may not be included. The bottom case corresponds to the decision not to include u in the event list, which requires that all of its children are included, and as a result any of its grandchildren may or may not be included.

We memoize using the vertices of the tree, storing $MPE(u)$ on vertex u itself, for each $u \in T$. We evaluate in a postorder traversal. There are $O(n)$ subproblems, and computing $MPE(u)$ takes $O(1 + |r(u)| + |g(u)|)$ time, where the $O(1)$ to account for work when $r(u) = g(u) = \emptyset$. Every employee is the child and grandchild of at most one vertex, so the total runtime is

$$O\left(\sum_{u \in T} (1 + |r(u)| + |g(u)|)\right) = O(n).$$

Solution: (An alternate recurrence.) We define two mutual functions as follows. Let $Include(u)$ (resp., $Exclude(u)$) be the minimum total price of a suitable guest list among the employees in the subtree rooted at employee u , where u is included (resp., not included). The functions are described

by the following recurrences:

$$\begin{aligned} \text{Include}(u) &= p(u) + \sum_{v \in r(u)} \max\{\text{Include}(v), \text{Exclude}(v)\} \\ \text{Exclude}(u) &= \sum_{v \in r(u)} \text{Include}(v) \end{aligned}$$

where, in each recurrence, the empty sum has value 0 (i.e., when an employee v has $r(v) = \emptyset$). To solve the given problem, we compute $\max\{\text{Include}(c), \text{Exclude}(c)\}$, where c is the CEO.

We memoize using the vertices of the tree, storing $\text{Include}(u), \text{Exclude}(u)$ on vertex u itself, for each $u \in T$. We evaluate in a postorder traversal. There are $O(n)$ subproblems, and computing $\text{Include}(u), \text{Exclude}(u)$ takes $O(1 + |r(u)|)$ time, where the $O(1)$ to account for work when $r(u) = \emptyset$. Every employee is the child of at most one vertex (exactly one for all nodes but the CEO), so the total runtime is

$$O\left(\sum_{u \in T} (1 + |r(u)|)\right) = O(n).$$

Problem 4

Freddie is the owner of Freddie's Patterned Mousepads (FPM), the premier vendor on Etsy for custom-sized neoprene mousepads. No matter how small or how large a mousepad you need (in whole units), FPM has you covered. Every one of their mousepads features the same world-renowned signature pattern: down-sloping stripes in Duke Navy Blue (hex code #012169) on solid grey.

To make the mousepads, Freddie begins with a single large rectangular $a \times b$ neoprene sheet, for some integers $a, b \geq 1$. Using his cutting machine, he can make either a vertical or horizontal cut across the sheet, dividing it into two smaller rectangular sheets. After a sequence of cuts, he has a pile of various rectangular sheets, which he finishes into mousepads by stitching up the sides then posting them for sale on his Etsy store. However, the prices his customers are willing to pay for different sized mousepads seem to be completely arbitrary; for example, a 1×7 mousepad may sell for \$37 and a 2×7 mousepad may only sell for \$4. In particular, because of the pattern, a 1×7 mousepad and 7×1 mousepad look different and may sell for wildly different prices. Overwhelmed, Freddie is asking for your help to figure out the most money he can make from a single $a \times b$ neoprene sheet.

A template file, `mousepads.py`, is provided on Canvas. Design and implement an **iterative dynamic programming** algorithm as `MAXREVENUE(a, b, P[0..a][0..b])` in `mousepads.py` that solves Freddie's problem, where the first two parameters a, b denote the $a \times b$ dimensions of the initial neoprene sheet and $P[0..a][0..b]$ is a 2D array such that $P[i, j]$ stores the price for which an $i \times j$ mousepad will sell, for any integers $0 \leq i \leq a$ and $1 \leq j \leq b$. P is such that $P[0][j] = P[i][0] = 0$ for all i, j —Freddie cannot sell a mousepad with zero width or height! For full credit, the running time of your algorithm should be within constant factors of a reference solution with asymptotic runtime $O(ab(a + b))$.

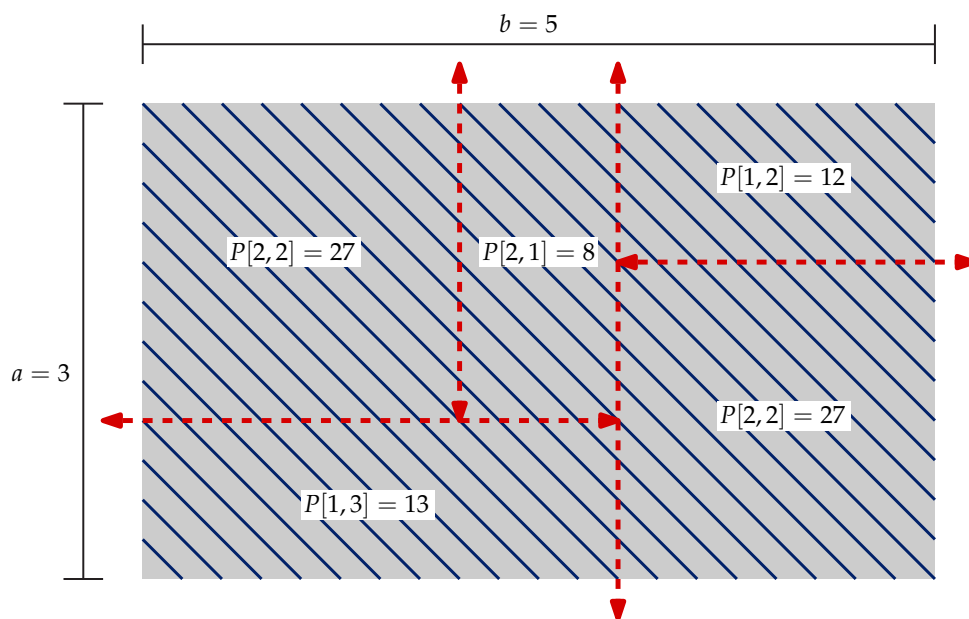


Figure 2. Example of four cuts on an initial 3×5 sheet, starting with the vertical cut across the entire sheet. The resulting five mousepads sell for $27 + 8 + 12 + 13 + 27 = 87$ dollars, following the given price array $P[0..3][0..5]$ below, which is the best possible for these prices.

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 12 & 13 & 3 & 7 \\ 0 & 8 & 27 & 30 & 11 & 9 \\ 0 & 8 & 7 & 0 & 8 & 16 \end{bmatrix}$$

Solution: Instead of providing our reference implementation, here is 0-indexed (Python-like) pseudocode.

```
MAX-REVENUE( $a, b, P$ ):
  if  $a = 0$  or  $b = 0$ :
    return 0
   $M[0..a][0..b] \leftarrow$  init. array with zeroes
  for row from 1 to  $a$ :
    for col from 1 to  $b$ :
      best-row-cut  $\leftarrow 0$ 
      for  $i$  from 1 to row:
        top  $\leftarrow M[i][row]$ 
        bottom  $\leftarrow M[row - i][col]$ 
        if best-row-cut  $< (top + bottom)$ :
          best-row-cut  $\leftarrow top + bottom$ 
      best-col-cut  $\leftarrow 0$ 
      for  $j$  from 1 to col:
        left  $\leftarrow M[row][j]$ 
        right  $\leftarrow M[row][col - j]$ 
        if best-col-cut  $< (left + right)$ :
          best-col-cut  $\leftarrow left + right$ 
      no-cut  $\leftarrow P[row][col]$ 
      best  $\leftarrow \max(\text{no-cut}, \text{best-row-cut}, \text{best-col-cut})$ 
       $M[row][col] \leftarrow \text{best}$ 
  return  $M[a][b]$ 
```